มหาวิทยาลัยมหิดล
Mahidol University

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วนโดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 4 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 4 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.

Mahidol University
Faculty of Information
and Communication Technology

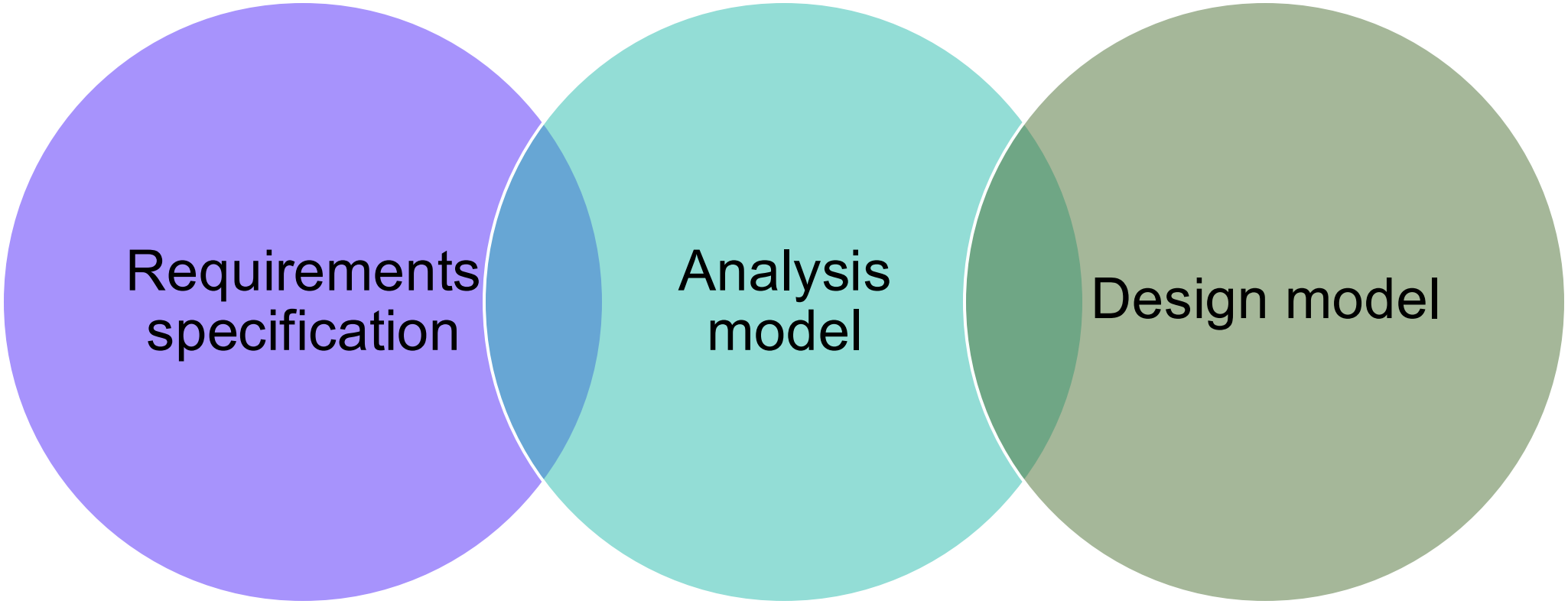
ITDS261 Introduction to Software Engineering

Lecture 5 – Requirements analysis with scenario-based modeling I

Topics

- Essential of use case diagrams
- Use case modeling techniques
- Use case modeling applications to requirements analysis

A Bridge



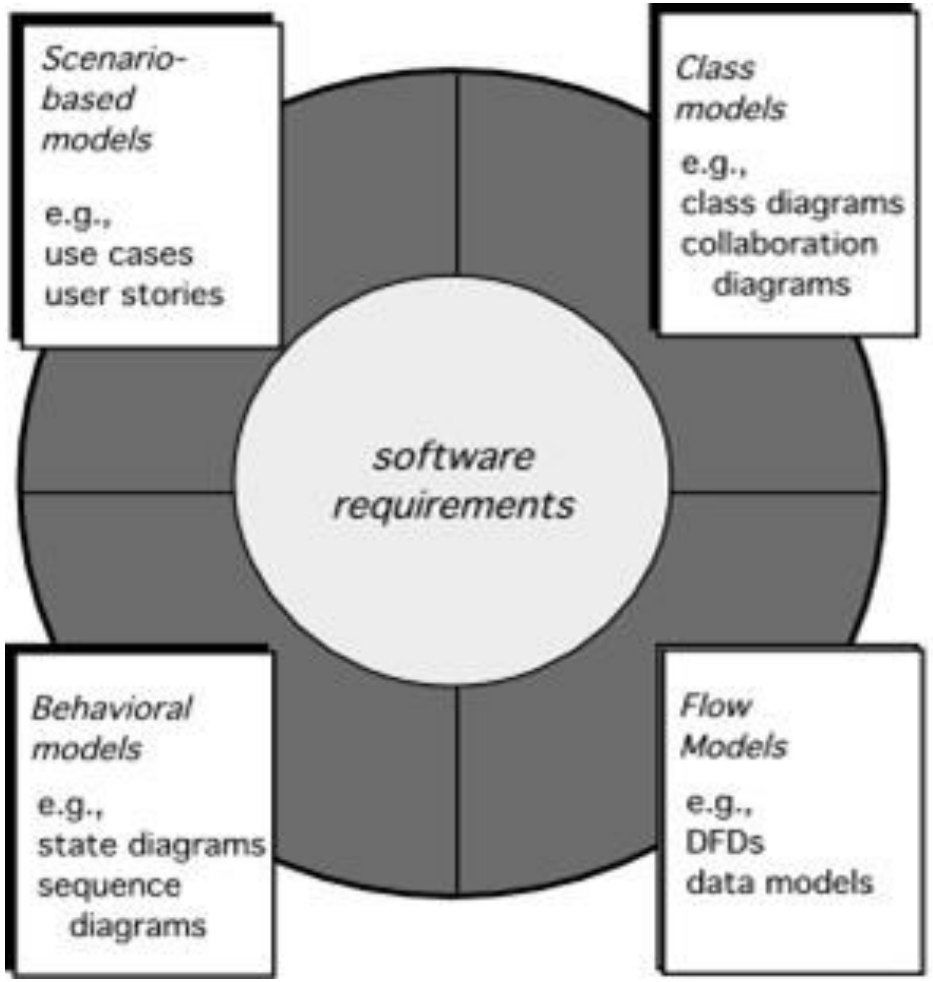
Requirements Analysis

- To specify software's operational characteristics
  - Elaborate on basic requirements established in requirements elicitation
- To indicate software's interface with other system elements
  - Build models that depict user scenarios, functional activities, relationship of classes and data flow
- To establish constraints that software must meet
- Usually performed by Business Analyst (BA)

Objectives of Requirements Analysis

- Describe what the customer requires
- Establish a basis for the creation of a software design
- Devise a set of requirements that can be validated once the software is built

Requirements Modeling Approaches



- ***Scenario-based models:*** depict requirements from the users' point of view (i.e., actors)
- ***Class models:*** represent object-oriented classes, attributes, operations and their relationships to achieve system requirements
- ***Flow models:*** represent the movement of data showing how it is transformed between processes in the system
- ***Behavioral models:*** depict how the software behaves in response to events
- ***Data models:*** depict the information domain for the problem (e.g., ER diagram)

Reference: Pressman, R.S. (2015) *Software engineering: A practitioner's approach*. New York, NY: McGraw-Hill Education.

Icon from Freepik

Scenario-based Models

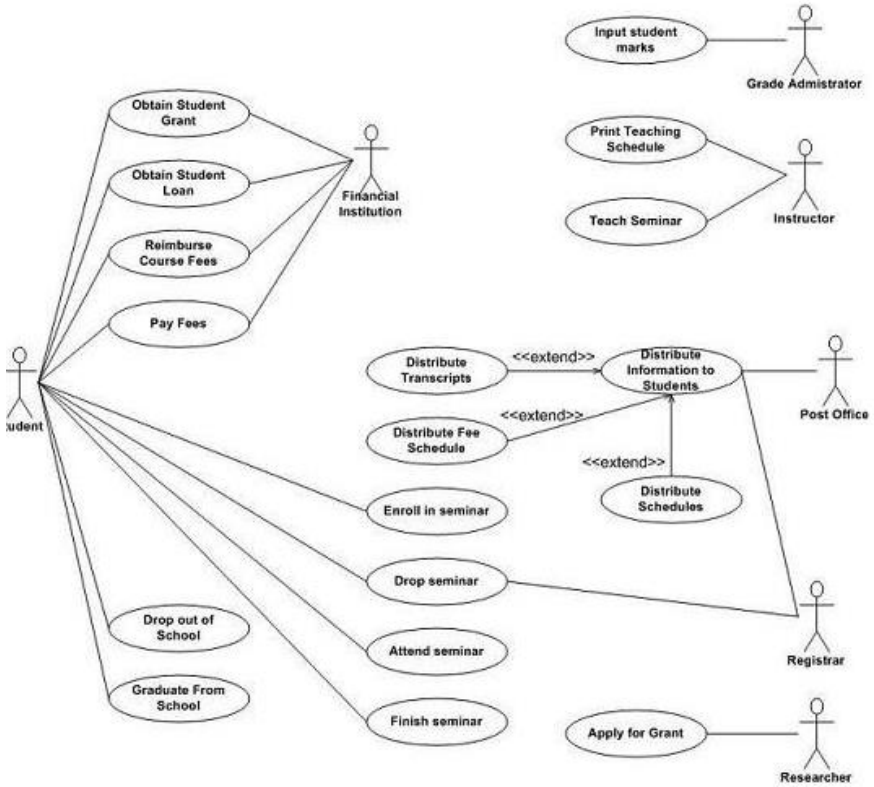
- Describe how actors will interact with the system in specific scenarios
- Focus on functional requirements from actors' point of view
- Useful for validating requirements with stakeholders
- Common technique -> Use case modelling

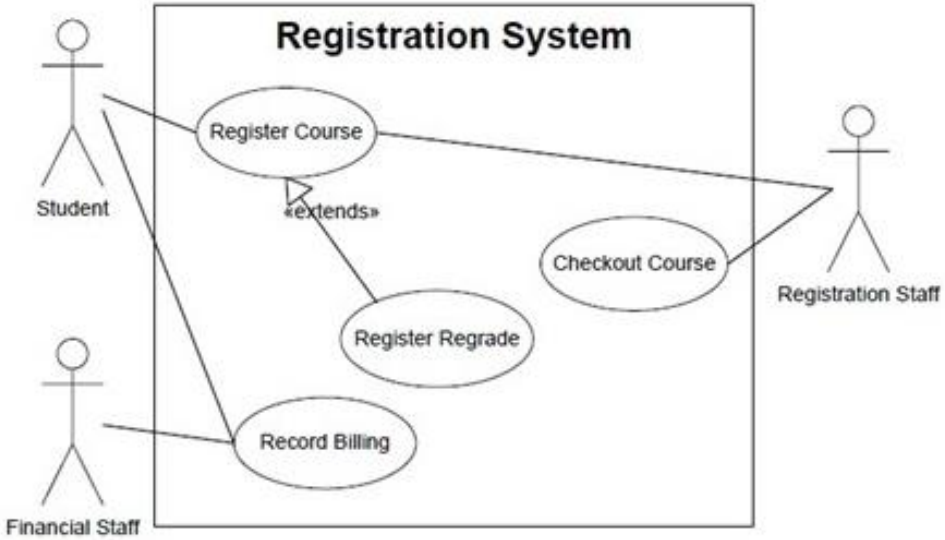
Steps for Requirements Analysis and Modeling

1) Conducting functional requirements analysis to understand the structure of the problem domain.
2) Breaking the structure into parts with the intention of understanding the parts' nature, inter-relationships, and their functions
3) Specifying the system's behaviors and functional requirements from the viewpoint of stakeholders
4) Writing, developing, Refining use cases diagrams and narratives
5) Confirming that the requirements models meet the needs of all stakeholders

Scenario-Based Modeling

- "**Use-cases** are simply an aid to defining what exists outside the system (**actors**) and what should be performed by the system (**use cases**)." Ivar Jacobson





What to Write About?

- Requirements gathering meetings, and other requirements engineering mechanisms are used to:
  - identify stakeholders
  - define the scope of the problem
  - specify overall operational goals
  - establish priorities
  - outline all known functional requirements, and
  - describe the things (objects) that will be manipulated by the system
- To begin developing a set of use cases, list the functions or activities performed by a specific actor

System Concepts for Use Case Modeling

Use case – a behaviourally related sequence of steps (scenario), both automated and manual, for the purpose of completing a single business task

- Description of system functions from the perspective of external users in terminology they understand

Use case diagram – a diagram that depicts the interactions between the system and external systems and users

- graphically describes who will use the system and in what ways the user expects to interact with the system

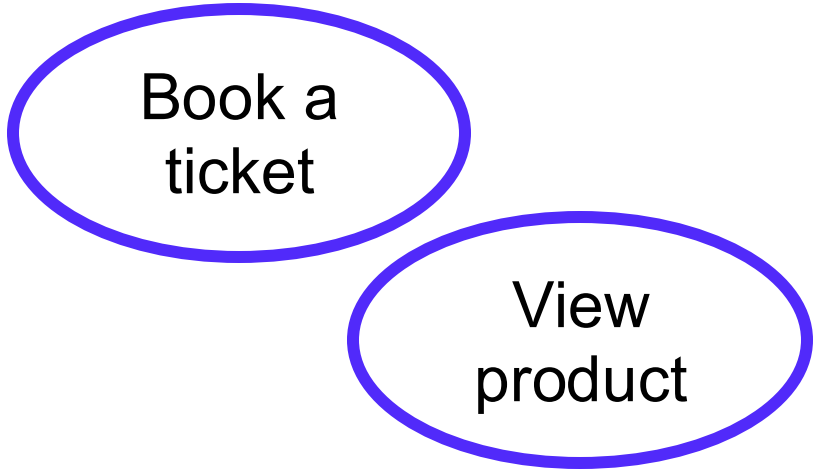
Use case narrative – a textual description of the business event and how the user will interact with the system to accomplish the task

Basic Use Case Symbols

Use case

Use an ellipse

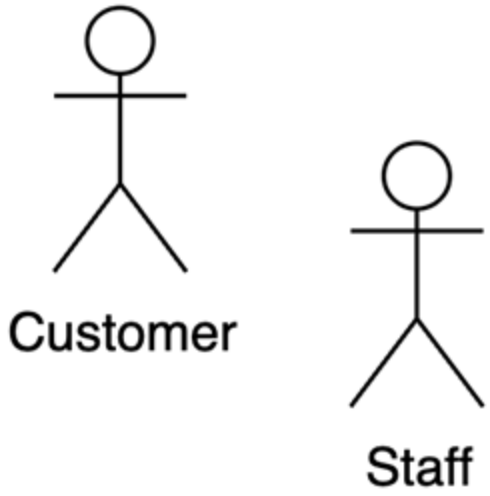
Begin with “verb”



Actor

Use a stickman symbol

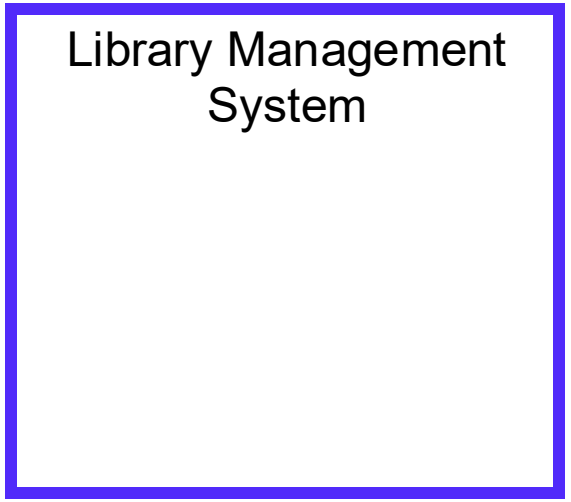
Label with “actor”



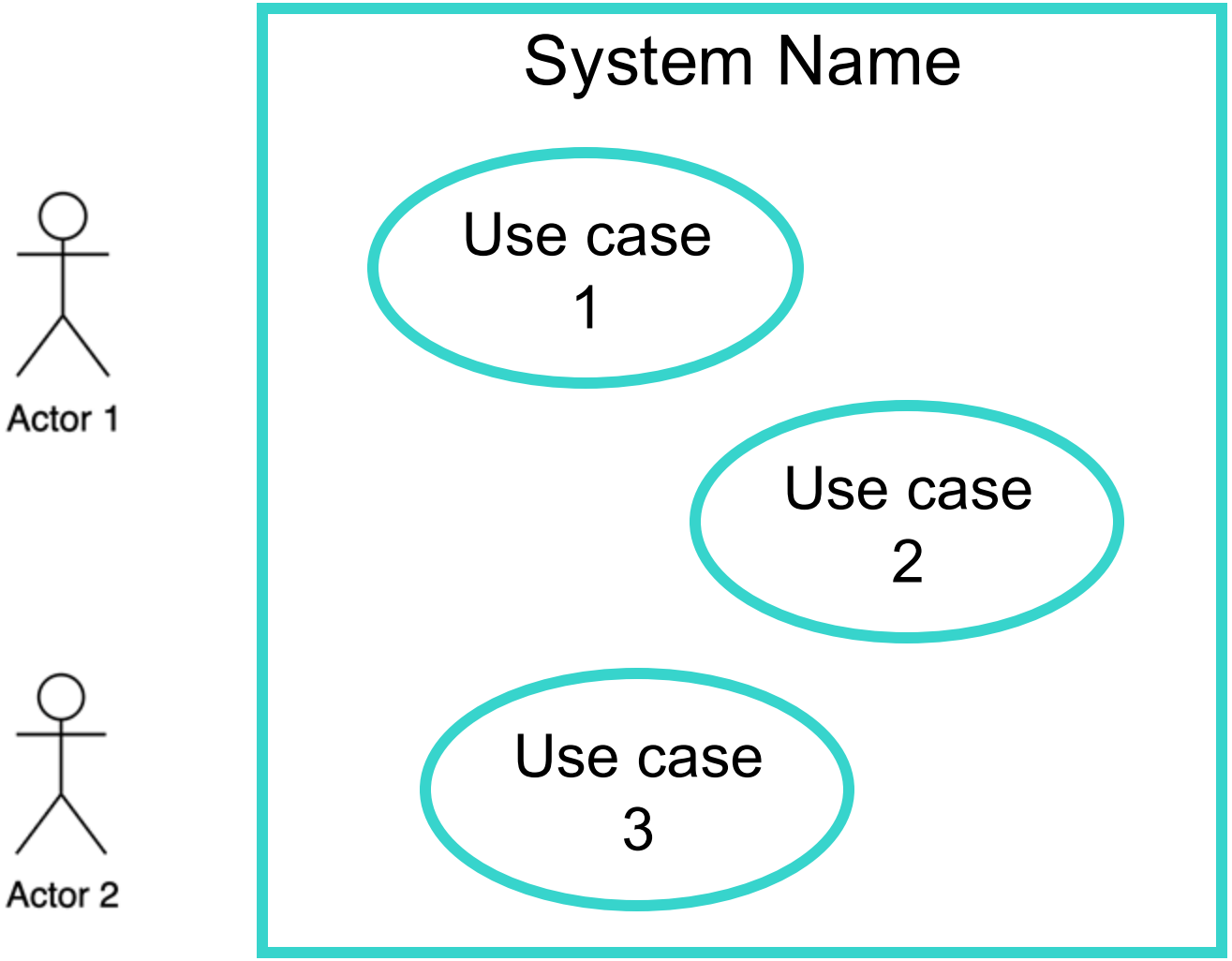
System boundary

Use a box

Label with “system name”



Three Components

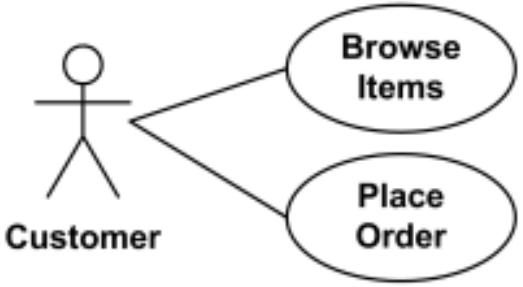


Use Case Relationships

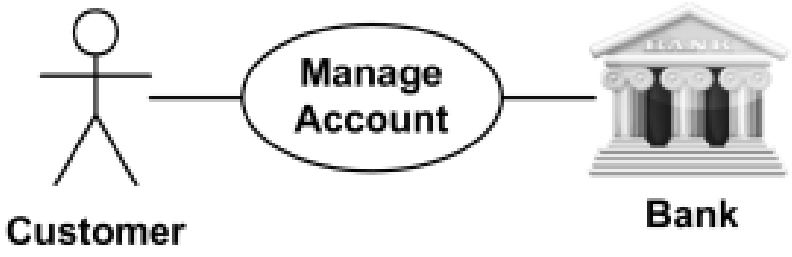
- High-level relationship
  - **Association** – represents an interaction between an actor and a use case
- Detailed relationships
  - Between actors
    - **Inheritance** – a relationship where common behavior of multiple actors is moved to a shared abstract actor to avoid redundancy
  - Between use cases
    - **Includes** – a relationship that simplifies large use case by extracting common behaviours
    - **Extends** – a relationship that provides an optional behaviour from the base use case.

Use Case Relationships: Association

- **Association** – represents an interaction between an actor and a use case

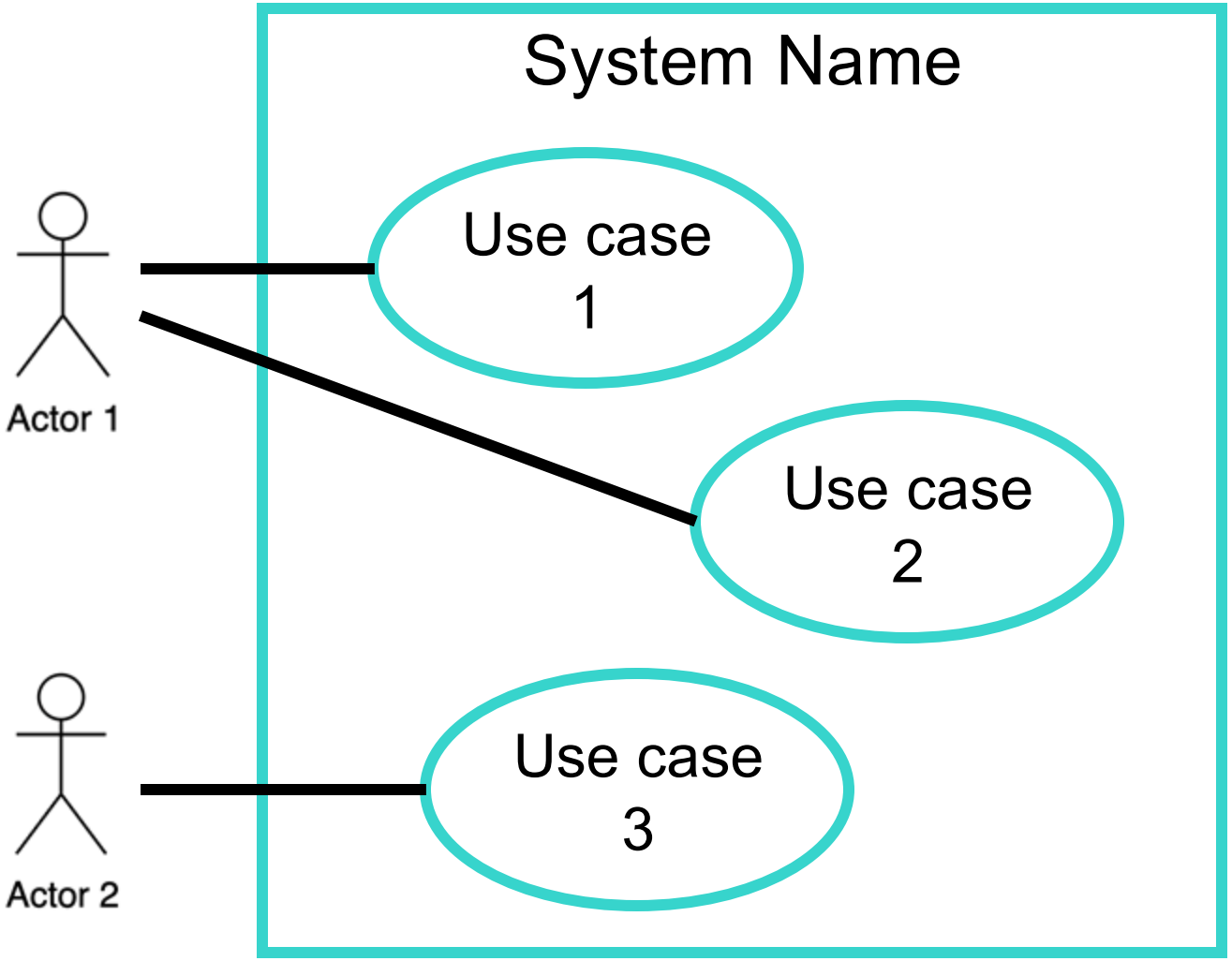


Customer actor is associated with two use cases - Browse Items and Place Order.



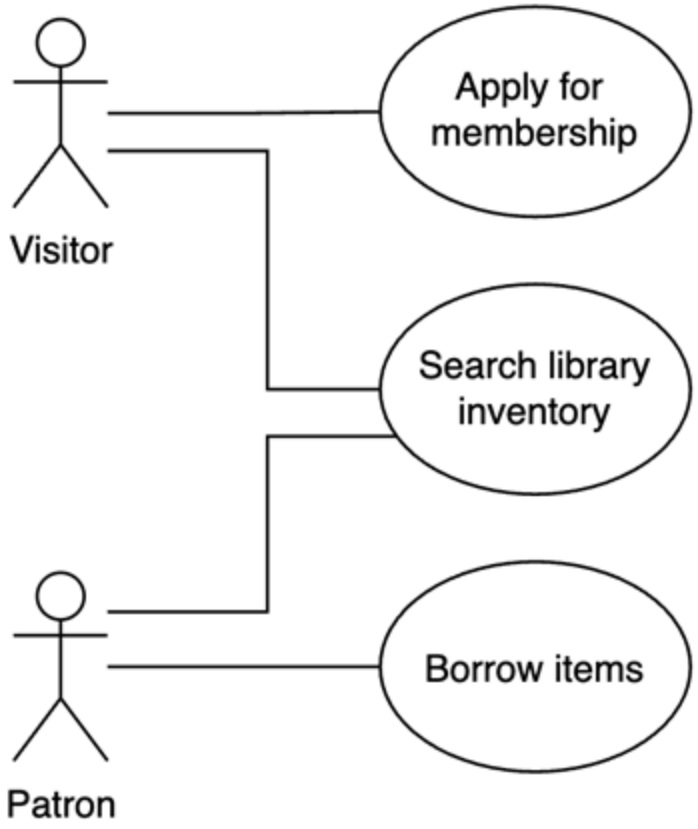
Manage Account use case is associated with Customer and Bank actors.

Use Case Diagram with Relationships



Use Case Relationships: Inheritance

- **Inheritance** – a relationship where common behavior of multiple actors is moved to a shared abstract actor to avoid redundancy.



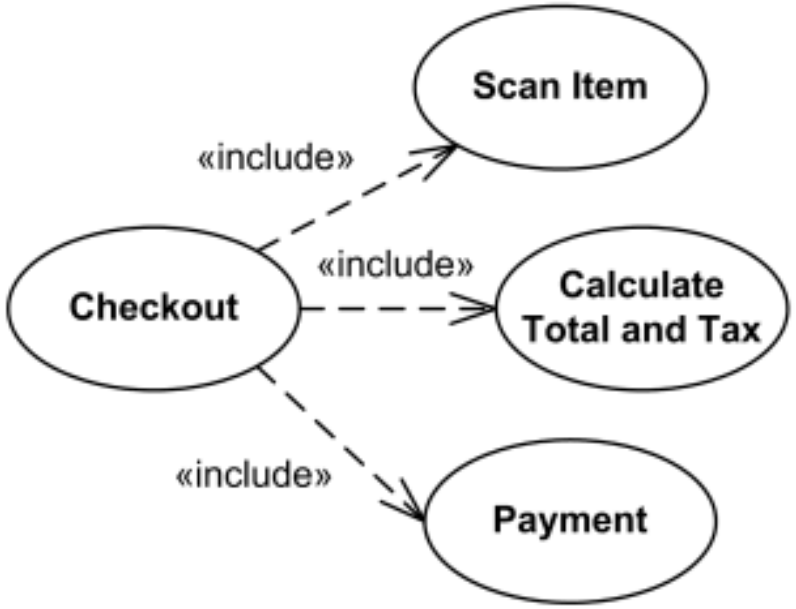
Before

After

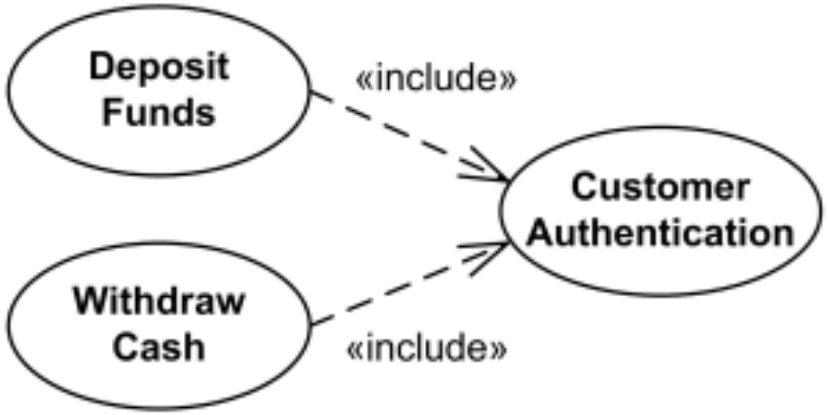
Use Case Relationships: Includes

- **Includes** – a relationship that simplifies large use case by extracting common behaviours.

Depicted as arrow beginning at original use case and pointing to use case it is using.
Labeled <<includes>>



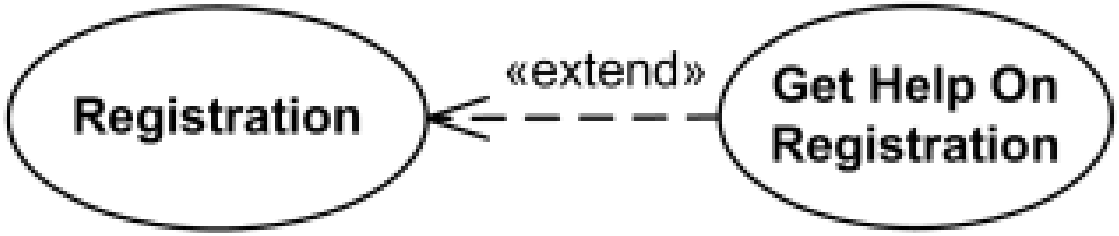
Checkout use case includes several use cases - Scan Item, Calculate Total and Tax, and Payment



***Deposit Funds** and **Withdraw Cash** use cases include **Customer Authentication** use case.*

Use Case Relationships: Extends

- **Extends** – a relationship that provides an optional behaviour from the base use case.



Depicted as arrow beginning at the extending use case and pointing to base use case it is extending.

Labeled <<extends>>

Registration *use case is complete and meaningful on its own.*

It could be extended with optional
Get Help On Registration *use case.*

Developing a Use Case (1)

1) **Identify the system boundary**
   - Define what system you are modeling.
   - Draw a box (system boundary) to represent the scope of the system.
2) **Identify the actors**
   - Determine who will interact with the system.
   - Actors can be primary
     (directly use the system to achieve goals, e.g., customer)
     or secondary (supporting role, e.g., a payment service).
3) **Identify use cases**
   - List the main tasks or goals that actors want to achieve with the system.
   - These should represent functional requirements
     (e.g., “Book Ticket”, “Generate Report”).

Developing a Use Case (2)

4) **Establish relationships between actors and use cases**
   - Connect actors to the use cases they initiate or participate in.
   - Consider types of relationships.
5) **Draw a use case diagram**
   - Place actors outside the system boundary.
   - Place use cases inside the system boundary.
   - Show associations and relationships clearly.
6) **Review and refine**
   - Check if all functional requirements are covered.
   - Validate with stakeholders to confirm completeness and accuracy.

Use Case Narrative

- A detailed textual description of how a system interacts with external entities (actors) to achieve a specific goal.
- Provides a step-by-step explanation of how a user or another system interacts with the software to accomplish a task.
- Usually expressed in a tabular format

Key Components of a Use Case Narrative

Use Case Name:	A clear, descriptive title. The name must be consistent with the one in a use case diagram.
Primary Actor:	A user or system that interacts with the use case.
Goal in Context:	The objective or purpose of the use case.
Precondition:	Conditions that must be met before the use case can begin.
Trigger:	The event that initiates the use case.
Scenario:	The typical sequence of interactions between the actor and the system. Sometimes it is called “main flow” or “basic flow”
Exception:	What happens if something goes wrong, such as an error or invalid input.
Postcondition:	The state of the system after the use case is completed.

Exercise: Zoo Ticketing System

Use Case Diagram

Use Case Name:	
Primary Actor:	
Goal in Context:	
Precondition:	
Trigger:	
Scenario:	
Exception:	
Postcondition:	

Use Case Name:	
Primary Actor:	
Goal in Context:	
Precondition:	
Trigger:	
Scenario:	
Exception:	
Postcondition:	